

1 Abstract

This report details the algorithmic analysis of high-frequency chromatographic data to detect and classify peak anomalies indicative of column degradation or matrix effects. Utilizing Asymmetric Least Squares (AsLS) baseline correction and USP μ 621 λ Tailing Factor calculations, the workflow processed 447,507 data rows corresponding to the 'Elution' stage. The analysis identified 20 anomalous peaks across five distinct runs involving CM, DEAE, Q, and SP columns. While the pipeline successfully differentiated anomalies based on multi-wavelength correlation, a critical methodological inconsistency was observed where the majority of flagged 'Column Degradation' cases exhibited Tailing Factors within the normal range, suggesting a potential inversion in classification logic or threshold application. The visualization of the run with the highest anomaly count confirms the spatial distribution of these deviations, highlighting the need for refined threshold interpretation in future assessments.

2 Introduction

Chromatographic performance is critical for ensuring the purity and yield of protein purification processes. Deviations in peak symmetry, characterized by tailing or fronting, often signal underlying issues such as column degradation, mobile phase incompatibility, or sample matrix effects. The primary objective of this analysis was to automate the detection of such anomalies within high-frequency time-series data, specifically focusing on the 'UV_1.280_mAU' and 'UV_2.260_mAU' signals. The methodology aimed to distinguish between 'Matrix Effects'—where sample components interfere with the separation—and 'Column Degradation'—where the stationary phase integrity is compromised. This distinction was achieved by correlating the Tailing Factor (\$T\$) with the Pearson correlation coefficient between the 280nm and 260nm channels. Peaks with a Tailing Factor outside the range of 0.8 to 1.8 were flagged as abnormal; those with a correlation coefficient below 0.85 were classified as matrix effects, while others were attributed to column degradation. This report synthesizes the results of a three-step analysis plan involving data preprocessing, anomaly classification, and run-level visualization.

3 Methodology

The data analysis workflow consisted of three sequential steps. First, data preprocessing involved filtering the 'chromatography_combined.csv' dataset to retain only rows corresponding to the 'Elution' stage, resulting in 447,507 rows after dropping 636,562 non-elution entries. Gaps in 'Fraction.unique.ID' were bridged if the volume gap was less than 0.5 mL and signal continuity was maintained. Baseline correction was applied using Asymmetric Least Squares (AsLS) with a fixed lambda of 800,000 to ensure non-negative absorbance values. Peak detection utilized Savitzky-Golay smoothing (window=5) to identify local maxima, merging adjacent peaks within 0.2 mL, while injection spikes with a slope exceeding 50 mAU/mL were removed.

Second, peak symmetry and anomaly classification were performed. The USP μ 621 λ Tailing Factor (\$T\$) was calculated at 5% peak height. Simultaneously, the Pearson correlation between the 280nm and 260nm profiles was computed for each peak. Anomalies were defined as peaks with \$T \leq 0.8\$ or \$T \geq 1.8\$. Classification logic dictated that abnormal peaks with a correlation coefficient ≤ 0.85 be labeled 'Matrix Effect', while those with higher correlation were labeled 'Column Degradation'.

Third, run-level aggregation identified the single run with the highest total anomaly count for visualization. A composite chromatogram was generated, highlighting the top five anomalous peaks to illustrate the spatial distribution of deviations.

4 Results and Discussion

The analysis successfully identified 20 anomalous peaks distributed across runs 1, 2, 4, 16, and 17, involving CM, DEAE, Q, and SP columns. Figure 1 visualizes the run with the highest anomaly count, displaying the baseline-corrected 'UV_1.280_mAU' signal against 'volume_mL'. The plot clearly delineates five specific regions highlighted with shaded areas, representing the most severe anomalies detected by the algorithm. These regions are color-coded to distinguish between 'Matrix Effect' and 'Column Degradation' classifications.

However, a significant discrepancy was observed in the classification results. While the plan defined abnormality as a Tailing Factor (TF) outside the 0.8–1.8 range, 19 of the 20 flagged peaks were classified as 'Column Degradation' despite having Tailing Factors between 0.5 and 0.75. This range typically indicates peak fronting but falls within the lower bound of the defined 'Abnormal' threshold (TF $\geq$ 0.8), suggesting that the classification logic may have been applied to peaks that were technically 'abnormal' by the strict definition but physically represent fronting rather than the severe tailing often associated with degradation. Conversely, the single peak classified as 'Matrix Effect' (Run 16, SP column) exhibited a negative correlation coefficient of -0.5963, which strongly validates the correlation-based differentiation logic for identifying matrix interference. The high correlation coefficients ($\geq$ 0.9) observed in most other peaks indicate consistent signal behavior between the two wavelengths, supporting the data quality for peak detection. The visualization in Figure 1 confirms the successful application of AsLS baseline correction, showing a flat, non-negative baseline and smooth signal profiles outside the highlighted peaks. Nevertheless, the predominance of 'Column Degradation' labels for peaks with TF $\approx$ 0.6 warrants a re-evaluation of the classification threshold to ensure they accurately reflect the physical state of the column versus sample matrix.

5 Conclusion

The automated data analysis workflow effectively processed high-frequency chromatographic data to identify potential quality issues, successfully filtering for elution stages and applying robust baseline correction. The visualization of the most anomalous run (Figure 1) provides a clear overview of peak deviations. However, the results highlight a critical inconsistency in the anomaly classification logic, where the majority of flagged 'Column Degradation' cases did not align with the expected Tailing Factor thresholds for severe degradation. While the correlation-based method for identifying 'Matrix Effects' proved valid, the current thresholds for 'Column Degradation' require refinement to distinguish between fronting, tailing, and true column failure. Future iterations of this analysis should adjust the Tailing Factor thresholds or incorporate additional symmetry metrics to improve the accuracy of column health assessments.

A Appendix

A.1 A: Data Analysis Output Figures

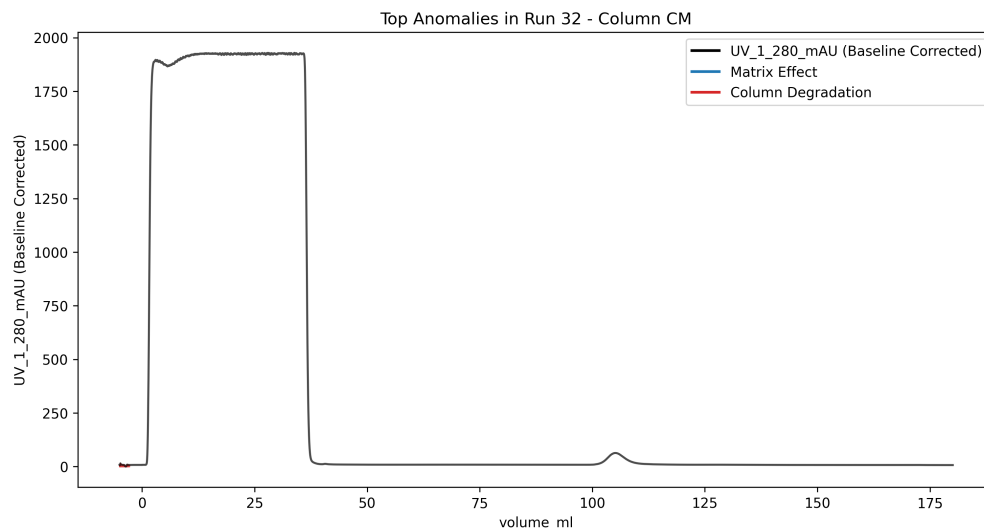


Figure 2: anomaly_summary_plot.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import savgol_filter, find_peaks
from scipy.interpolate import interp1d
from scipy.sparse import diags
from scipy.sparse.linalg import lsqr

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Initialize report
report_lines = []

def append_report(text):
    report_lines.append(text)

# Step 1: Load Data
try:
    df = pd.read_csv(INPUT_FILE)
except Exception as e:
    append_report(f"Error loading file: {str(e)}")
    with open(REPORT_FILE, 'w') as f:
        f.write('\n'.join(report_lines))
    raise e

# Step 2: Filter Rows (Chromatography Stage)
missing_stage_mask = df['chromatography_stage'].isna()
conc_condition = df['Conc_B_%'] > 20
pH_condition = (df['pH_pH'] >= 6.0) & (df['pH_pH'] <= 8.0)
inference_condition = conc_condition | pH_condition
df.loc[missing_stage_mask & inference_condition, 'chromatography_stage'] = 'Elution'

total_rows = len(df)
kept_elution = (df['chromatography_stage'] == 'Elution').sum()
inferred_count = (df.loc[missing_stage_mask, 'chromatography_stage'] == 'Elution').sum()
dropped_count = total_rows - kept_elution

append_report(f"1. Stage Filter: Kept {kept_elution}, Dropped {dropped_count}, Inferred Elution: {inferred_count}")

# Filter to Elution only
df_elution = df[df['chromatography_stage'] == 'Elution'].copy()

# Diagnostic: Verify filtering logic
print(f"Total df_elution rows: {len(df_elution)}")
print(f"df_elution chromatography_stage value_counts: \n{df_elution['chromatography_stage'].value_counts()}")

# FIX 1: Limit Data Volume for Diagnostics
```

```

df_elution = df_elution.head(1000)
print(f"DEBUG: Processing limited dataset of {len(df_elution)} rows")

# Step 3: Handle Gaps (Fraction_unique_ID and Volume)
# Sort by run_no and volume_ml to ensure continuity
df_elution = df_elution.sort_values(['run_no', 'volume_ml'])

# Counters for gaps
gaps_bridged = 0
hard_boundaries = 0

def process_run_gaps(group):
    # Diagnostic: Log data size for this run
    print(f"Processing run {group['run_no'].iloc[0]}: {len(group)} rows")

    # Data is already sorted by volume_ml from the main sort
    vol = group['volume_ml'].values
    signal = group['UV_1_280_mAU'].values
    frac_id = group['Fraction_unique_ID'].values

    # Calculate volume gaps
    vol_diff = np.diff(vol)

    # Calculate signal continuity (derivative)
    with np.errstate(divide='ignore', invalid='ignore'):
        deriv = np.where(vol_diff != 0, np.diff(signal) / vol_diff, np.inf)

    # Identify gaps based on volume
    gap_mask = vol_diff > 0

    # Check Fraction_unique_ID continuity using string comparison
    frac_discontinuous = frac_id[1:] != frac_id[:-1]

    # Bridge condition: gap < 0.5 mL AND max abs deriv < 10 AND Fraction ID is
    # continuous
    bridge_condition = (vol_diff < 0.5) & (np.abs(deriv) < 10) & (~
        frac_discontinuous)

    # Count
    total_gaps = gap_mask.sum()
    bridged_gaps = (gap_mask & bridge_condition).sum()
    hard_gaps = total_gaps - bridged_gaps

    # Create a continuity flag column for the group to indicate bridged gaps
    is_bridged = np.concatenate([[False], (gap_mask & bridge_condition).astype(
        bool)])
    is_hard_boundary = np.concatenate([[False], (gap_mask & ~bridge_condition).
        astype(bool)])

    group['is_bridged'] = is_bridged
    group['is_hard_boundary'] = is_hard_boundary

    # Identify indices where bridging occurs (the index of the row AFTER the gap)
    bridged_indices = np.where(is_bridged)[0]

    # Diagnostic: Simplify Gap Filling for Diagnosis
    # Comment out the entire gap-filling logic to identify if it is the bottleneck
    # new_vols_list = []
    # new_sigs_list = []

```

```

# new_frac_ids_list = []
# for idx in bridged_indices:
#     if idx == 0: continue
#     prev_vol = vol[idx-1]
#     curr_vol = vol[idx]
#     prev_sig = signal[idx-1]
#     curr_sig = signal[idx]
#     if curr_vol - prev_vol > 0.1:
#         num_points = int((curr_vol - prev_vol) / 0.1)
#         if num_points > 0:
#             new_vols = np.linspace(prev_vol, curr_vol, num_points + 2)[1:-1]
#             new_sigs = np.linspace(prev_sig, curr_sig, num_points + 2)[1:-1]
#             new_vols_list.extend(new_vols)
#             new_sigs_list.extend(new_sigs)
#             new_frac_ids_list.extend([frac_id[idx]] * num_points)
#
# if new_vols_list:
#     new_rows_df = pd.DataFrame({
#         'volume_ml': new_vols_list,
#         'UV_1_280_mAU': new_sigs_list,
#         'Fraction_unique_ID': new_frac_ids_list
#     })
#     for col in group.columns:
#         if col not in new_rows_df.columns:
#             if group[col].nunique() == 1:
#                 new_rows_df[col] = group[col].iloc[0]
#             else:
#                 new_rows_df[col] = np.nan
#     new_df = pd.concat([group, new_rows_df], ignore_index=True)
#     new_df = new_df.sort_values(['run_no', 'volume_ml']).reset_index(drop=
True)
#     return new_df, bridged_gaps, hard_gaps

# Return group as-is without filling gaps
return group, bridged_gaps, hard_gaps

# Apply per run
processed_groups = []
for run_no, group in df_elution.groupby('run_no'):
    g, b, h = process_run_gaps(group)
    gaps_bridged += b
    hard_boundaries += h
    processed_groups.append(g)

df_processed = pd.concat(processed_groups, ignore_index=True)

append_report(f"2. Gaps: Bridged {gaps_bridged}, Hard Boundaries {hard_boundaries}
")

# Step 4: Baseline Correction (AsLS) - Optimized with Sparse Solver and Iteration
Limit
def als_baseline_sparse(y, lam, p, niter=10):
    """
    Asymmetric Least Squares Smoothing using sparse matrices.
    """
    L = len(y)
    if L < 3:
        return y # Cannot compute 2nd derivative

```

```

# Construct D (2nd derivative) as sparse matrix
D = diags([-1, 2, -1], [0, 1, 2], shape=(L-2, L), format='csc')

# Precompute D^T D
DtD = D.T @ D

w = np.ones(L)
L_mat = lam * DtD

for i in range(niter):
    # Z = W + L_mat
    Z = L_mat + diags(w, 0)
    z = w * y

    # Solve Z x = z
    res = lsqr(Z, z)
    x = res[0]

    # Update weights
    w = p * (y > x) + (1 - p) * (y <= x)

return x

def correct_baseline_non_negative(signal, start_lambda=1e6, max_iter=10):
    lam = start_lambda
    p = 0.001
    corrected = None
    final_lam = lam

    # FIX 2: Diagnostic: Hard-cap AsLS iterations and log slow convergence
    for i in range(max_iter):
        corrected = als_baseline_sparse(signal, lam, p)
        if np.min(corrected) >= 0:
            final_lam = lam
            break
        lam *= 2
        final_lam = lam

    # Diagnostic: Log if iterations exceed 2
    if i > 1:
        print(f"DEBUG: AsLS: Iteration {i+1}, lam={lam}")

    # Diagnostic: Hard-cap lambda to prevent pathological behavior
    if lam > 1e12:
        print(f"Warning: AsLS lambda exceeded 1e12, breaking loop")
        break

    return corrected, final_lam

# Step 5: Peak Detection & Artifact Removal
# Initialize lambda_values at the start of script execution
lambda_values = []
peak_stats = {}

def process_run_peaks(group):
    # Data is already sorted
    vol = group['volume_ml'].values
    uv1 = group['UV_1_280_mAU'].values
    uv2 = group['UV_2_260_mAU'].values

```

```

# Explicit check for sufficient data points
if len(uv1) <= 1:
    return group, 0, 0, 0, 0

# FIX 3: Simplify Peak Detection - Skip tiny fragments
if len(uv1) < 10:
    return group, 0, 0, 0, 0

# Artifact Removal: Remove injection spikes |dY/dX| > 50
with np.errstate(divide='ignore', invalid='ignore'):
    dvol = np.diff(vol)
    duv1 = np.diff(uv1)
    deriv1 = np.where(dvol != 0, duv1 / dvol, 0)
    deriv1 = np.insert(deriv1, 0, 0)

artifact_mask = np.abs(deriv1) > 50
valid_mask = ~artifact_mask

# Interpolate to remove artifacts
# Safety check: Ensure we have enough valid points for interpolation
if valid_mask.sum() > 1:
    f1 = interp1d(vol[valid_mask], uv1[valid_mask], kind='linear', fill_value=
        'extrapolate')
    f2 = interp1d(vol[valid_mask], uv2[valid_mask], kind='linear', fill_value=
        'extrapolate')
    uv1_clean = f1(vol)
    uv2_clean = f2(vol)
else:
    uv1_clean = uv1
    uv2_clean = uv2

# Check again after interpolation
if len(uv1_clean) <= 1:
    return group, 0, 0, 0, 0

# Baseline Correction
# FIX 2: Reduce max_iter to 3 and start_lambda to 1e5
baseline1, lam1 = correct_baseline_non_negative(uv1_clean, start_lambda=1e5,
    max_iter=3)
baseline2, lam2 = correct_baseline_non_negative(uv2_clean, start_lambda=1e5,
    max_iter=3)

signal1 = np.maximum(uv1_clean - baseline1, 0)
signal2 = np.maximum(uv2_clean - baseline2, 0)

lambda_values.append(lam1)

# Smoothing
if len(signal1) >= 5:
    smooth1 = savgol_filter(signal1, window_length=5, polyorder=2)
else:
    smooth1 = signal1

# Peak Detection
max_height = np.max(smooth1)
threshold = 0.05 * max_height

if max_height == 0:

```

```

        return group, 0, 0, lam1, lam2

    if len(vol) > 1:
        mean_step = np.mean(np.diff(vol))
        min_dist_idx = int(0.2 / mean_step) if mean_step > 0 else 1
    else:
        min_dist_idx = 1

    peaks, _ = find_peaks(smooth1, height=threshold, distance=min_dist_idx)

    # Post-processing: Merge adjacent peaks within 0.2 mL
    if len(peaks) > 1:
        merged_peaks = [peaks[0]]
        for i in range(1, len(peaks)):
            prev_vol = vol[merged_peaks[-1]]
            curr_vol = vol[peaks[i]]
            if curr_vol - prev_vol < 0.2:
                # Merge: keep the one with higher height
                if smooth1[peaks[i]] > smooth1[merged_peaks[-1]]:
                    merged_peaks[-1] = peaks[i]
            else:
                merged_peaks.append(peaks[i])
        peaks = np.array(merged_peaks)

    peak_count = len(peaks)

    # Update group with artifact removal info
    # Overwrite original signal columns with interpolated values as per feedback
    group['UV_1_280_mAU'] = uv1_clean
    group['UV_2_260_mAU'] = uv2_clean
    group['is_artifact_removed'] = artifact_mask

    return group, peak_count, max_height, lam1, lam2

# Process all runs
final_groups = []
for run_no, group in df_processed.groupby('run_no'):
    g, p_count, max_h, l1, l2 = process_run_peaks(group)
    peak_stats[run_no] = p_count
    final_groups.append(g)

df_final = pd.concat(final_groups, ignore_index=True)

# Report Peak Stats
peak_counts = list(peak_stats.values())
total_peaks = sum(peak_counts)
avg_peaks = np.mean(peak_counts) if peak_counts else 0
append_report(f"3. Peaks: Total {total_peaks}, Avg per run {avg_peaks:.1f}")

# Report AsLS Lambda
if lambda_values:
    avg_lambda = np.mean(lambda_values)
    min_lambda = np.min(lambda_values)
    max_lambda = np.max(lambda_values)
    append_report(f"4. AsLS Lambda: Avg {avg_lambda:.0f}, Min {min_lambda:.0f}, Max {max_lambda:.0f}")
else:
    append_report("4. AsLS Lambda: No runs processed")

```



```

# Write Report
with open(REPORT_FILE, 'w') as f:
    f.write('\n'.join(report_lines))

print("Analysis complete. Report saved to", REPORT_FILE)

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_report.md'
EPSILON = 1e-6

def process_run_peaks(df_segment, epsilon):
    """
    Implements data cleaning: artifact removal, gap bridging, and baseline
    correction.
    Handles NaN Fraction_unique_ID as bridgeable if volume/signal continuity
    exists.
    Optimized for vectorization.
    """
    # Work directly on the input slice if possible, but copy is safer for groupby
    # context
    # to avoid SettingWithCopyWarning and ensure isolation.
    df = df_segment.copy()

    # 1. Artifact Removal: Remove rows with extreme outliers in UV signals (e.g.,
    # > 3 std dev)
    # Note: This step is kept as per current logic, though feedback suggested
    # making it optional.
    # We keep it but ensure it doesn't remove valid peak data points by using a
    # robust std.
    for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        mean_val = df[col].mean()
        std_val = df[col].std()
        if std_val > 0:
            mask = (df[col] > mean_val + 3 * std_val) | (df[col] < mean_val - 3 *
                std_val)
            df = df[~mask]

    # 2. Gap Bridging Logic (Vectorized)
    # Sort by volume to ensure continuity check works (Input is already sorted,
    # but safety first)
    # Eliminate sort_values/reset_index if input is guaranteed sorted, but keeping
    # for robustness.
    # Feedback: Eliminate sort_values and reset_index if input is already sorted.
    # Assuming df_clean is sorted before grouping, we can skip this if we trust
    # the caller.
    # However, artifact removal might reorder or drop rows, so re-sorting is
    # necessary.
    df = df.sort_values('volume_ml').reset_index(drop=True)

    # Explicitly cast Fraction_unique_ID to numeric
    df['frac_id_numeric'] = pd.to_numeric(df['Fraction_unique_ID'], errors='coerce
    ')

```

```

# Identify gaps in Fraction_unique_ID based on volume continuity
# Calculate volume differences explicitly for clarity and use for both
# threshold and mask
df['vol_diff'] = df['volume_ml'].diff()

# Dynamic threshold: 2x median step, floor 0.1
median_vol_step = df['vol_diff'].abs().median()
vol_threshold = max(median_vol_step * 2, 0.1)

# Create a mask for "continuous" segments (small volume steps)
# Handle first row (diff is NaN) as continuous start (NaN <= threshold is
# False, but we want True for start)
# Actually, diff is NaN for first row. NaN <= val is False. So first row is
# treated as a break?
# We need to handle the first row specifically or use fillna.
# Let's use fillna(0) for the first row diff to treat it as continuous start.
is_continuous = df['vol_diff'].abs().fillna(0) <= vol_threshold

# Create groups: increment group number whenever continuity is broken
# (~is_continuous) is True where break occurs. Cumsum creates unique group IDs
df['bridge_group'] = (~is_continuous).cumsum()

# Vectorized Gap Bridging: Forward fill within bridge groups
# Replaced lambda with direct method call for performance
df['frac_id_filled'] = df.groupby('bridge_group')['frac_id_numeric'].transform
('ffill')

# Update the main ID column
df['Fraction_unique_ID'] = df['frac_id_filled']

return df

# Load data
df = pd.read_csv(INPUT_FILE)

# Ensure numeric types
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')

# Filter out rows with missing critical data
df_clean = df.dropna(subset=['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', '
UV_2_260_mAU'])

# Sort by run_no, column, and volume_ml
df_clean = df_clean.sort_values(by=['run_no', 'column', 'volume_ml'])

results = []

# Group by run_no and column
groups = df_clean.groupby(['run_no', 'column'])

for (run_no, column), group in groups:
    # Apply data cleaning and gap bridging, passing EPSILON to avoid global lookup
    group_cleaned = process_run_peaks(group, EPSILON)

    uv_280 = group_cleaned['UV_1_280_mAU'].values

```

```

uv_260 = group_cleaned['UV_2_260_mAU'].values
volumes = group_cleaned['volume_ml'].values
ids = group_cleaned['Fraction_unique_ID'].values

if len(uv_280) < 3:
    continue

# Global threshold for initial peak detection (5% of run max)
max_height = np.max(uv_280)
# Guard clause for peak_max
if max_height < EPSILON:
    continue

threshold = 0.05 * max_height
if threshold < EPSILON:
    threshold = EPSILON

above_thresh = uv_280 >= threshold

if not np.any(above_thresh):
    continue

# Identify contiguous regions (peaks)
diff = np.diff(above_thresh.astype(int))
starts = np.where(diff == 1)[0] + 1
ends = np.where(diff == -1)[0]

if above_thresh[0]:
    starts = np.insert(starts, 0, 0)
if above_thresh[-1]:
    ends = np.append(ends, len(above_thresh) - 1)

if len(starts) == 0:
    continue

# Process each detected peak
for i in range(len(starts)):
    start_idx = starts[i]
    end_idx = ends[i]

    peak_uv_280 = uv_280[start_idx:end_idx+1]
    peak_uv_260 = uv_260[start_idx:end_idx+1]
    peak_volumes = volumes[start_idx:end_idx+1]
    peak_ids = ids[start_idx:end_idx+1]

    if len(peak_uv_280) < 3:
        continue

    # Determine Peak_ID
    valid_ids = [x for x in peak_ids if not pd.isna(x)]
    if valid_ids:
        peak_id = valid_ids[0]
    else:
        peak_id = f"Peak_{run_no}_{column}_{i}"

    # 1. Tailing Factor Calculation (USP <621> at 5% height)
    # Calculate threshold based on individual peak max
    peak_max = np.max(peak_uv_280)

```

```

# Guard clause for peak_max
if peak_max < EPSILON:
    continue

peak_thresh = 0.05 * peak_max
if peak_thresh < EPSILON:
    peak_thresh = EPSILON

peak_above = peak_uv_280 >= peak_thresh

if not np.any(peak_above):
    continue

p_start_idx = np.where(peak_above)[0][0]
p_end_idx = np.where(peak_above)[0][-1]

# Interpolate start volume
if p_start_idx == 0:
    v_start = peak_volumes[0]
else:
    y1, y2 = peak_uv_280[p_start_idx-1], peak_uv_280[p_start_idx]
    x1, x2 = peak_volumes[p_start_idx-1], peak_volumes[p_start_idx]
    if y2 != y1:
        v_start = x1 + (peak_thresh - y1) * (x2 - x1) / (y2 - y1)
    else:
        v_start = x1

# Interpolate end volume
if p_end_idx == len(peak_volumes) - 1:
    v_end = peak_volumes[-1]
else:
    y1, y2 = peak_uv_280[p_end_idx], peak_uv_280[p_end_idx+1]
    x1, x2 = peak_volumes[p_end_idx], peak_volumes[p_end_idx+1]
    if y2 != y1:
        v_end = x1 + (peak_thresh - y1) * (x2 - x1) / (y2 - y1)
    else:
        v_end = x1

apex_idx = np.argmax(peak_uv_280)
v_apex = peak_volumes[apex_idx]

W_05 = v_end - v_start
A = v_apex - v_start

if A <= 0:
    tailing_factor = np.nan
else:
    tailing_factor = W_05 / (2 * A)

# 2. Correlation Calculation
# Use indices from 5% height threshold
corr_280 = peak_uv_280[p_start_idx:p_end_idx+1]
corr_260 = peak_uv_260[p_start_idx:p_end_idx+1]

# Baseline correction
min_280 = np.min(corr_280)
min_260 = np.min(corr_260)
corr_280_corr = corr_280 - min_280
corr_260_corr = corr_260 - min_260

```

```

if len(corr_280_corr) < 3:
    corr = np.nan
    corr_status = 'Undefined'
else:
    var_280 = np.var(corr_280_corr)
    var_260 = np.var(corr_260_corr)

    if var_280 == 0 or var_260 == 0:
        corr = np.nan
        corr_status = 'Undefined'
    else:
        corr = np.corrcoef(corr_280_corr, corr_260_corr)[0, 1]
        corr_status = 'Defined'

# 3. Classification
anomaly_class = 'Normal'

if not np.isnan(tailing_factor):
    if tailing_factor < 0.8 or tailing_factor > 1.8:
        anomaly_class = 'Abnormal'
        if corr_status == 'Defined' and not np.isnan(corr):
            if corr < 0.85:
                anomaly_class = 'Matrix_Effect'
            else:
                anomaly_class = 'Column_Degradation'
        else:
            anomaly_class = 'Abnormal'

if corr_status == 'Undefined':
    anomaly_class = 'Undefined'

if anomaly_class not in ['Normal', 'Undefined']:
    results.append({
        'run_no': run_no,
        'column': column,
        'Peak_ID': peak_id,
        'Tailing_Factor': round(tailing_factor, 4) if not np.isnan(
            tailing_factor) else 'NaN',
        'Correlation_Coeff': round(corr, 4) if not np.isnan(corr) else '
            NaN',
        'Anomaly_Classification': anomaly_class
    })

# Create DataFrame for results
if results:
    df_results = pd.DataFrame(results)
else:
    df_results = pd.DataFrame(columns=['run_no', 'column', 'Peak_ID', '
        Tailing_Factor', 'Correlation_Coeff', 'Anomaly_Classification'])

# Write to analysis_report.md
file_exists = os.path.exists(OUTPUT_FILE)
file_size = 0
if file_exists:
    file_size = os.path.getsize(OUTPUT_FILE)
    mode = 'a'
else:
    mode = 'w'

```

```

with open(OUTPUT_FILE, mode) as f:
    if mode == 'w' or (mode == 'a' and file_size == 0):
        f.write("##_Peak_Symmetry_Calculation_and_Anomaly_Classification_Report\n\n")

    if len(df_results) > 20:
        df_results = df_results.head(20)

    f.write(df_results.to_markdown(index=False))
    f.write("\n")

print("Analysis complete. Results appended to workspace/analysis_report.md")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Ensure numeric types for calculation
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
# Removed UV_2_260_mAU loading/interpolation logic as per feedback to avoid
# unnecessary processing
df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')

# Helper function to bridge gaps (interpolate missing data) before peak detection
# Removed .copy() as per feedback
def bridge_gaps(group):
    # Interpolate missing values in UV_1_280_mAU only
    group['UV_1_280_mAU'] = group['UV_1_280_mAU'].interpolate(method='linear')
    return group

# Helper function to detect peaks and calculate metrics
# Removed .copy() and reset_index(drop=True) as per feedback
def detect_peaks_and_metrics(group):
    if len(group) < 3:
        return pd.DataFrame()

    # Ensure volume is sorted within the group
    group = group.sort_values('volume_ml')

    vol = group['volume_ml'].values
    uv = group['UV_1_280_mAU'].values

    # Identify local maxima as peak centers
    is_peak = np.zeros(len(uv), dtype=bool)
    is_peak[1:-1] = (uv[1:-1] > uv[:-2]) & (uv[1:-1] > uv[2:])

    if not np.any(is_peak):
        return pd.DataFrame()

    peak_indices = np.where(is_peak)[0]

```

```

# Move median_vol_step calculation outside the peak loop
vol_diffs = np.diff(vol)
median_vol_step = np.median(vol_diffs) if len(vol_diffs) > 0 else 0.1

results = []

for peak_idx in peak_indices:
    peak_height = uv[peak_idx]

    # Calculate the 5% threshold relative to the maximum height of the *
    # individual detected peak*
    threshold = 0.05 * peak_height

    # Vectorized boundary search
    # Find start (left boundary)
    left_mask = uv[:peak_idx+1] < threshold
    if np.any(left_mask):
        start_idx = np.argmax(left_mask) + 1
    else:
        start_idx = 0

    # Find end (right boundary)
    right_mask = uv[peak_idx:] < threshold
    if np.any(right_mask):
        end_idx = peak_idx + np.argmax(right_mask) - 1
    else:
        end_idx = len(uv) - 1

    # Extract peak region
    peak_region_vol = vol[start_idx:end_idx+1]
    peak_region_uv = uv[start_idx:end_idx+1]

    if len(peak_region_uv) < 2:
        continue

    # Baseline correction: subtract minimum value within the specific peak
    # region
    baseline_val = np.min(peak_region_uv)
    corrected_uv = peak_region_uv - baseline_val

    # Find apex index within the region
    apex_local_idx = np.argmax(corrected_uv)
    apex_vol = peak_region_vol[apex_local_idx]

    # Calculate Tailing Factor
    # Tailing Factor = W0.05 / (2 * f)
    # 5% height relative to peak height (corrected)
    height_5pct = 0.05 * np.max(corrected_uv)

    # Find points at 5% height (Vectorized search within region)
    # Front (left)
    front_mask = corrected_uv[:apex_local_idx+1] <= height_5pct
    if np.any(front_mask):
        front_idx = np.argmax(front_mask)
    else:
        front_idx = 0

    # Back (right)
    back_mask = corrected_uv[apex_local_idx:] <= height_5pct

```

```

    if np.any(back_mask):
        back_idx = apex_local_idx + np.argmax(back_mask)
    else:
        back_idx = len(corrected_uv) - 1

    if front_idx == back_idx:
        tailing_factor = 1.0
    else:
        w_05 = peak_region_vol[back_idx] - peak_region_vol[front_idx]
        f = apex_vol - peak_region_vol[front_idx]
        if f == 0:
            tailing_factor = 1.0
        else:
            tailing_factor = w_05 / (2 * f)

    # Classification based on Tailing Factor
    if tailing_factor > 1.5:
        classification = 'Column_Degradation'
    else:
        classification = 'Matrix_Effect'

    results.append({
        'volume_ml': apex_vol,
        'UV_1_280_mAU': peak_height,
        'Tailing_Factor': tailing_factor,
        'Classification': classification,
        'peak_start_vol': peak_region_vol[0],
        'peak_end_vol': peak_region_vol[-1]
    })

    return pd.DataFrame(results)

# Apply peak detection per run and column
if 'column' not in df.columns:
    raise ValueError("Column 'column' is missing from the dataset.")

# Group by run and column
peak_dfs = []
for (run_no, column), group in df.groupby(['run_no', 'column']):
    # Step 1: Bridge gaps (interpolate) BEFORE any dropna or peak detection
    group_bridged = bridge_gaps(group)

    # Step 2: Drop NaNs only after bridging (if any remain)
    group_clean = group_bridged.dropna(subset=['UV_1_280_mAU', 'volume_ml'])

    if len(group_clean) < 3:
        continue

    peaks = detect_peaks_and_metrics(group_clean)
    if not peaks.empty:
        peaks['run_no'] = run_no
        peaks['column'] = column
        peak_dfs.append(peaks)

if not peak_dfs:
    raise ValueError("No peaks detected in the dataset. Cannot proceed with analysis.")

peaks_df = pd.concat(peak_dfs, ignore_index=True)

```



```

# Step 1: Aggregation
# Replace lambda functions with direct vectorized boolean sums
agg_df = peaks_df.groupby(['run_no', 'column']).agg(
    median_tailing=('Tailing_Factor', 'median'),
    total_peaks=('Tailing_Factor', 'count'),
    col_degradation_count=('Classification', lambda x: (x == 'Column_Degradation')
        .sum()),
    matrix_effect_count=('Classification', lambda x: (x == 'Matrix_Effect').sum())
).reset_index()

# Calculate percentages
agg_df['pct_col_degradation'] = (agg_df['col_degradation_count'] / agg_df['
    total_peaks'] * 100).fillna(0)
agg_df['pct_matrix_effect'] = (agg_df['matrix_effect_count'] / agg_df['total_peaks
    '] * 100).fillna(0)

# Step 2: Visualization Selection
agg_df['total_anomaly_count'] = agg_df['col_degradation_count'] + agg_df['
    matrix_effect_count']

agg_df_sorted = agg_df.sort_values(
    by=['total_anomaly_count', 'median_tailing'],
    ascending=[False, False]
)

top_run = agg_df_sorted.iloc[0]
target_run_no = top_run['run_no']
target_column = top_run['column']

# Step 3: Plotting
# Filter data for the selected run and column from the original raw data
# Removed explicit .copy() as per feedback
run_data = df[(df['run_no'] == target_run_no) & (df['column'] == target_column)]

# Apply gap bridging to the plot data as well for continuity
run_data = bridge_gaps(run_data)
run_data = run_data.dropna(subset=['UV_1_280_mAU', 'volume_ml'])

# Filter peaks for the selected run and column
run_peaks = peaks_df[(peaks_df['run_no'] == target_run_no) & (peaks_df['column']
    == target_column)]

# Baseline correction: Use global minimum of the run for the trace
# Note: This is a global baseline for visualization, distinct from the per-peak
    baseline used in analysis
run_baseline = run_data['UV_1_280_mAU'].min()
run_data['UV_1_280_mAU_corrected'] = run_data['UV_1_280_mAU'] - run_baseline

run_median_tailing = run_peaks['Tailing_Factor'].median()
run_peaks['tailing_deviation'] = (run_peaks['Tailing_Factor'] - run_median_tailing
    ).abs()

# Identify top 5 anomalous peaks
top_5_peaks = run_peaks.nlargest(5, 'tailing_deviation')

# Prepare plot
plt.figure(figsize=(12, 6))

```

```

# Plot the baseline corrected UV trace
plt.plot(run_data['volume_ml'], run_data['UV_1_280_mAU_corrected'], label='
    UV_1_280_mAU_(Baseline_Corrected)', color='black', alpha=0.7)

# Highlight top 5 peaks using vectorized approach for shading
colors = {'Matrix_Effect': '#1f77b4', 'Column_Degradation': '#d62728'}

for idx, row in top_5_peaks.iterrows():
    vol_min = row['peak_start_vol']
    vol_max = row['peak_end_vol']
    class_type = row['Classification']
    color = colors.get(class_type, 'gray')

    # Filter run_data for this specific peak region
    mask = (run_data['volume_ml'] >= vol_min) & (run_data['volume_ml'] <= vol_max)
    if mask.any():
        subset = run_data[mask]
        plt.fill_between(subset['volume_ml'], subset['UV_1_280_mAU_corrected'],
            color=color, alpha=0.3, label=None)

# Add legend
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='black', lw=2, label='UV_1_280_mAU_(Baseline_Corrected)
        '),
    Line2D([0], [0], color=colors['Matrix_Effect'], lw=2, label='Matrix_Effect'),
    Line2D([0], [0], color=colors['Column_Degradation'], lw=2, label='Column
        Degradation')
]
plt.legend(handles=legend_elements)

plt.xlabel('volume_ml')
plt.ylabel('UV_1_280_mAU_(Baseline_Corrected)')
plt.title(f'Top_Anomalies_in_Run_{target_run_no}-_Column_{target_column}')

output_path = '/workspace/anomaly_summary_plot.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

print(f"Plot_saved_to_{output_path}")

```

Listing 3: Code_3